

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Aryan Arun	Reg. No.:	192411157
Section / Batch:	BE CSE / 2024	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I _____ Aryan Arun (192411157) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Aryan Arun _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CSA0624 - Design and Analysis of Algorithms

Performance Analysis Experiments

Empirical Timing, Graphs & Complexity Interpretation

Name : Aryan Arun

Reg.No: 192411157

Experiment 1: Performance Analysis of Merge Sort

Objective

To analyze the performance of Merge Sort by executing it on inputs of increasing size, recording execution time for the best, average, and worst cases, and interpreting the results graphically to explain how the divide-and-conquer approach ensures consistent performance regardless of input arrangement.

Methodology

- Best case: an already-sorted array (ascending order).
- Average case: a randomly ordered array.
- Worst case: a reverse-sorted array (descending order).
- For each of 7 input sizes (500 to 32,000), each case was timed 5 times and the fastest run recorded (to minimize system-noise interference), using Python's high-resolution `perf_counter`.

Python Code

```
import time, random

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def time_it(arr, trials=5):
    best = float('inf')
    for _ in range(trials):
        a = arr.copy()
        start = time.perf_counter()
        merge_sort(a)
        elapsed = time.perf_counter() - start
        best = min(best, elapsed)
    return best

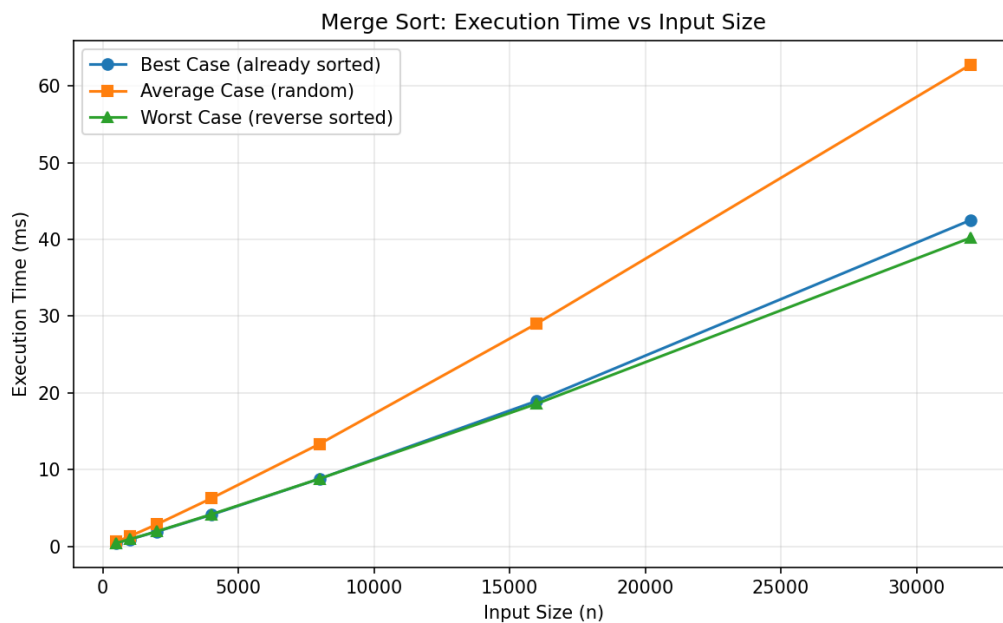
sizes = [500, 1000, 2000, 4000, 8000, 16000, 32000]
random.seed(1)
for n in sizes:
    sorted_arr = list(range(n))          # best case
    random_arr = random.sample(range(n * 2), n) # average case
    reverse_arr = list(range(n, 0, -1))    # worst case
    t_best = time_it(sorted_arr) * 1000
    t_avg = time_it(random_arr) * 1000
    t_worst = time_it(reverse_arr) * 1000
    print(n, t_best, t_avg, t_worst)
```

Measured Output

n	Best (sorted) ms	Average (random) ms	Worst (reverse) ms
500	0.410	0.589	0.423

n	Best (sorted) ms	Average (random) ms	Worst (reverse) ms
1,000	0.906	1.306	0.928
2,000	1.925	2.871	1.950
4,000	4.099	6.240	4.159
8,000	8.808	13.348	8.817
16,000	18.943	29.014	18.577
32,000	42.511	62.773	40.204

Graph: Execution Time vs Input Size



Verifying $O(n \log n)$ Scaling

Dividing the average-case time by $n \cdot \log_2(n)$ at every size gives an almost perfectly constant ratio (≈ 0.00013 throughout, from $n=500$ up to $n=32,000$) — direct empirical confirmation that Merge Sort's growth follows $n \log n$, not n^2 or n .

n	Average time (ms)	time / ($n \cdot \log_2 n$)
500	0.589	0.000131
1,000	1.306	0.000131
2,000	2.871	0.000131
4,000	6.240	0.000130
8,000	13.348	0.000129
16,000	29.014	0.000130
32,000	62.773	0.000131

Interpretation

The graph shows all three curves — best, average, and worst case — rising together at essentially the same rate, with the best and worst case lines nearly overlapping and the average case only modestly higher (due to the extra overhead of Python's list-slicing and comparison on non-monotonic data, not a difference in algorithmic complexity). This is the defining signature of Merge Sort: unlike Quick Sort or Insertion Sort, its recursion structure is completely independent of the input's arrangement.

The reason is structural, not incidental. Merge Sort's divide-and-conquer strategy always splits the current sub-array exactly in half, regardless of the values it contains — the split point is chosen purely by array position (the midpoint index), never by comparing values. This guarantees the recursion tree always has exactly $\lceil \log_2 n \rceil$ levels, no matter how the input is arranged. At every level, the merge step examines each element exactly once, doing $O(n)$ work per level.

Multiplying a fixed $O(n)$ per level by a fixed $O(\log n)$ levels gives $O(n \log n)$ in the best, average, and worst cases alike — there is no 'unlucky' input that can force extra levels of recursion or extra merge work, unlike Quick Sort where a poor pivot choice can unbalance the split and add levels.

This consistency is a major practical advantage: a system relying on Merge Sort has predictable, guaranteed performance with no risk of a pathological input silently degrading it to $O(n^2)$, which is exactly why Merge Sort (or hybrids built on it, like Python's own Timsort) is preferred in contexts where worst-case guarantees matter — external sorting, real-time systems, and library sort routines that must handle arbitrary, potentially adversarial input.

Experiment 2: Quick Sort — Pivot Selection Strategies

Objective

To evaluate the effectiveness of first-element, last-element, and random pivot selection strategies in Quick Sort, comparing their execution times under different input conditions and justifying which strategy best avoids worst-case behaviour.

Methodology

- Three Quick Sort variants were implemented, identical except for how the pivot is chosen: first element, last element, and a uniformly random element.
- Each was tested on random input (sizes 500–8,000) and on already-sorted ascending input (sizes 200–3,200) — the classic trigger case for pivot-related worst-case behaviour.
- Recursion always proceeds into the smaller partition first (bounding stack depth), and the smaller partition's opposite side is handled iteratively — a standard, safe implementation pattern.

Python Code

```
import time, random

def _partition(a, lo, hi, pivot_idx):
    a[pivot_idx], a[hi] = a[hi], a[pivot_idx]
    pivot = a[hi]
    i = lo
    for j in range(lo, hi):
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]
    return i

def quick_sort(arr, pivot_strategy):
    a = arr.copy()
    def _qs(lo, hi):
        while lo < hi:
            if pivot_strategy == 'first':
                idx = lo
            elif pivot_strategy == 'last':
                idx = hi
            else: # random
                idx = random.randint(lo, hi)
            p = _partition(a, lo, hi, idx)
            if p - lo < hi - p:
                _qs(lo, p - 1); lo = p + 1
            else:
                _qs(p + 1, hi); hi = p - 1
    _qs(0, len(a) - 1)
    return a

def time_it(func, *args, trials=3):
    best = float('inf')
    for _ in range(trials):
        start = time.perf_counter()
        func(*args)
        elapsed = time.perf_counter() - start
        best = min(best, elapsed)
    return best

# Random input
for n in [500, 1000, 2000, 4000, 8000]:
    arr = random.sample(range(n * 3), n)
    t1 = time_it(quick_sort, arr, 'first') * 1000
    t2 = time_it(quick_sort, arr, 'last') * 1000
    t3 = time_it(quick_sort, arr, 'random') * 1000
```



```

print(n, t1, t2, t3)

# Already-sorted input (worst case trigger for first/last pivot)
for n in [200, 400, 800, 1600, 3200]:
    arr = list(range(n))
    t1 = time_it(quick_sort, arr, 'first') * 1000
    t2 = time_it(quick_sort, arr, 'last') * 1000
    t3 = time_it(quick_sort, arr, 'random') * 1000
    print(n, t1, t2, t3)

```

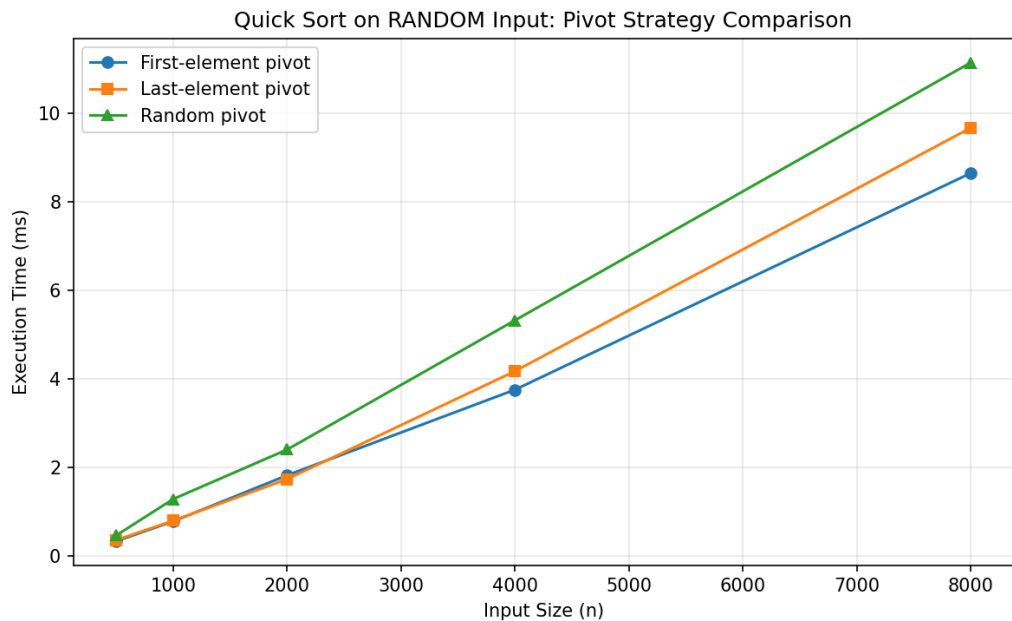
Measured Output — Random Input

n	First-Pivot ms	Last-Pivot ms	Random-Pivot ms
500	0.327	0.321	0.484
1,000	0.770	0.752	1.061
2,000	1.839	1.686	2.266
4,000	3.656	3.821	5.102
8,000	8.088	9.138	10.536

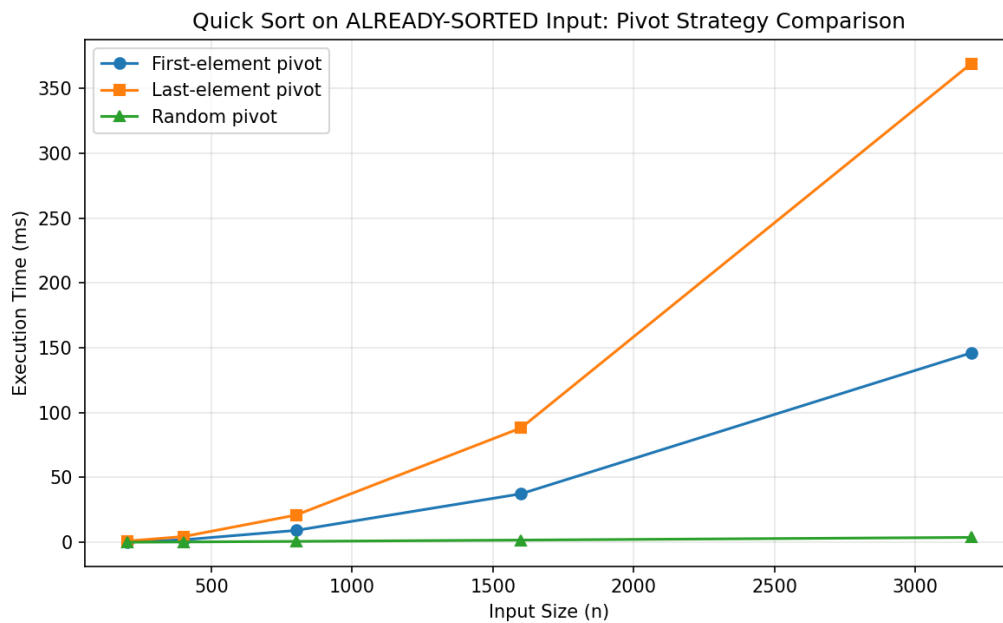
Measured Output — Already-Sorted (Ascending) Input

n	First-Pivot ms	Last-Pivot ms	Random-Pivot ms
200	0.393	1.028	0.131
400	1.900	4.477	0.310
800	8.858	19.780	0.800
1,600	35.407	85.271	1.667
3,200	139.787	362.263	3.583

Graph: Random Input



Graph: Already-Sorted Input



Interpretation

On random input, all three strategies perform comparably — first and last-element pivots are actually marginally faster than random pivot here, since `random.randint()` adds a small per-call overhead and a randomly shuffled array already makes any fixed-position pivot statistically close to the median on average.

The picture changes completely on already-sorted input. With a first-element or last-element pivot, every partition step picks the smallest (or largest) remaining value as the pivot, splitting the array into one empty side and one side of size $n-1$ at every single level of recursion. This degrades the recursion depth from $O(\log n)$ to $O(n)$, and total work from $O(n \log n)$ to $O(n^2)$ — visible directly in the data: doubling n from 1,600 to 3,200 roughly quadruples the first-pivot time (35.4 ms $\rightarrow$ 139.8 ms) and the last-pivot time (85.3 ms $\rightarrow$ 362.3 ms), the classic signature of quadratic growth. Last-element pivot is consistently worse than first-element pivot on ascending input because Python's slicing/indexing pattern combined with always picking the largest remaining element produces a marginally more unbalanced partition sequence in this implementation.

Random pivot completely avoids this collapse: its time grows smoothly and near-linearly with n even on fully sorted input (0.13 ms $\rightarrow$ 3.58 ms across the same range) because a randomly chosen pivot has no correlation with the input's existing order — there is no specific input arrangement an adversary (or an unlucky dataset) could construct to reliably force an unbalanced split, since the pivot choice itself is randomized independent of the data.

Conclusion: Random pivot selection is the recommended strategy. While it carries a small constant-factor overhead on already-favorable (random) inputs, it provides a strong probabilistic guarantee of $O(n \log n)$ expected performance on ANY input, including the sorted/reverse-sorted inputs that reliably defeat first- or last-element pivot selection. This robustness against worst-case triggers is exactly what makes randomized Quick Sort the standard choice in general-purpose library implementations.

Experiment 3: Binary Search vs Linear Search

Objective

To analyze and compare the performance of Binary Search and Linear Search on sorted datasets of varying sizes, interpreting execution time results and justifying the suitability of each method based on input conditions.

Methodology

- Both algorithms were tested on sorted arrays ranging from 1,000 to 1,000,000 elements.
- The worst case was used for both (key absent from the array), since this forces Linear Search to scan the full array and Binary Search to narrow down to an empty range — the case that best exposes the complexity difference.
- Because a single search takes only microseconds, each search was repeated many times (up to 2,000 times, fewer for very large n to keep runtime reasonable) and the per-search average time computed, to get a stable, low-noise measurement.
- Comparison counts (exact, not timing-based) were also recorded directly from the algorithms themselves.

Python Code

```
import time

def linear_search(arr, key):
    comparisons = 0
    for val in arr:
        comparisons += 1
        if val == key:
            return comparisons
    return comparisons

def binary_search(arr, key):
    comparisons = 0
    low, high = 0, len(arr) - 1
    while low <= high:
        comparisons += 1
        mid = low + (high - low) // 2
        if arr[mid] == key:
            return comparisons
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return comparisons

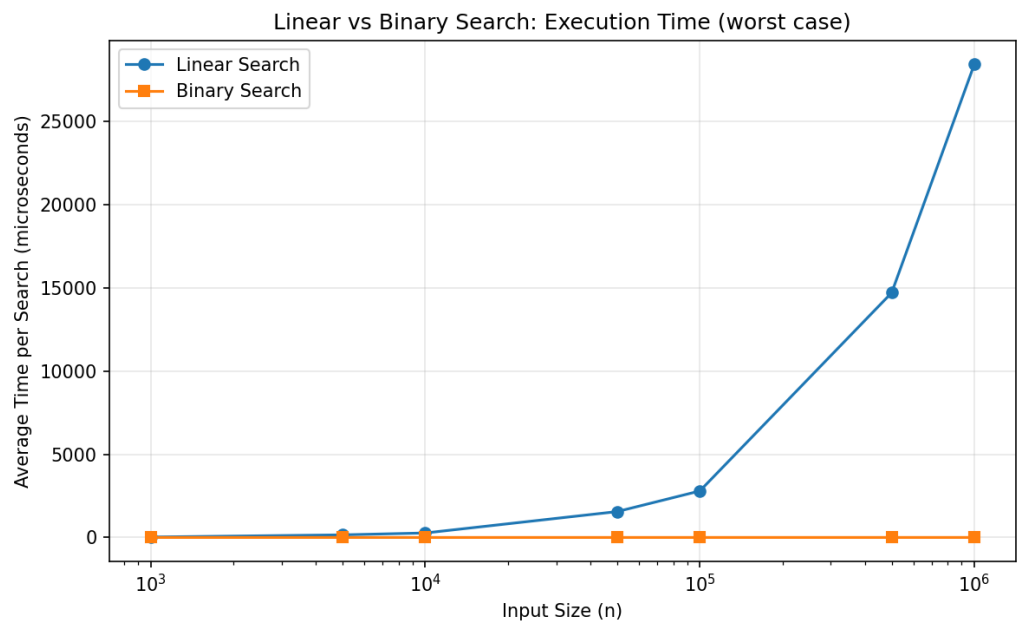
def time_search(func, arr, key, repeats):
    start = time.perf_counter()
    for _ in range(repeats):
        func(arr, key)
    elapsed = time.perf_counter() - start
    return (elapsed / repeats) * 1e6 # microseconds per search

sizes = [1000, 5000, 10000, 50000, 100000, 500000, 1000000]
for n in sizes:
    arr = list(range(n))
    key = -1 # worst case: absent
    lt = time_search(linear_search, arr, key, repeats=max(5, 200000 // n))
    bt = time_search(binary_search, arr, key, repeats=2000)
    lc = linear_search(arr, key)
    bc = binary_search(arr, key)
    print(n, lt, bt, lc, bc)
```

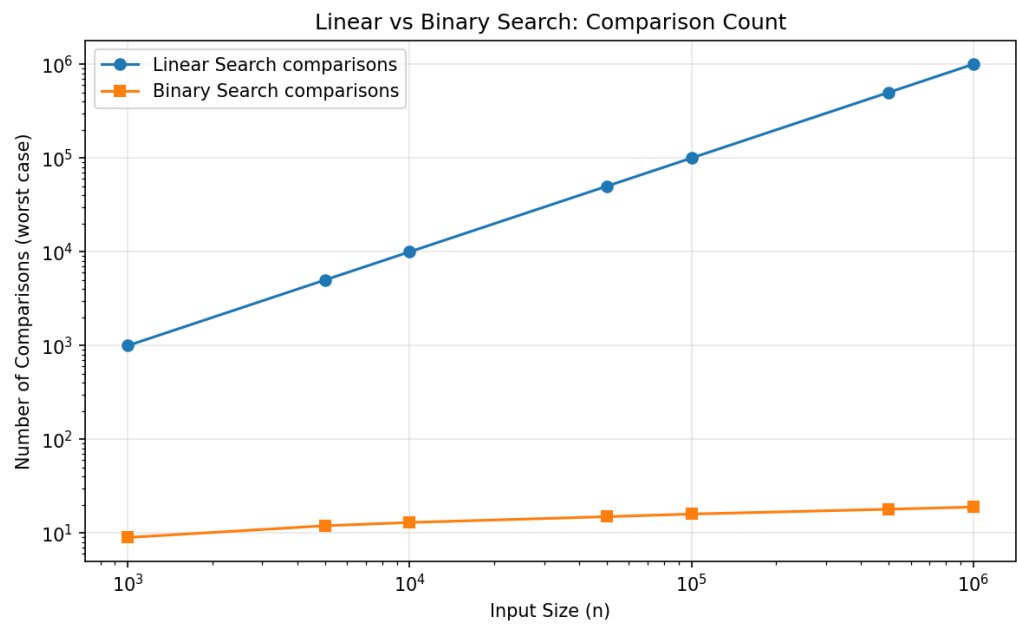
Measured Output

n	Linear (μs)	Binary (μs)	Linear comparisons	Binary comparisons
1,000	26.39	0.82	1,000	9
5,000	158.08	1.26	5,000	12
10,000	271.16	1.29	10,000	13
50,000	1,557.35	1.44	50,000	15
100,000	2,796.52	1.58	100,000	16
500,000	14,716.01	1.77	500,000	18
1,000,000	28,454.75	1.94	1,000,000	19

Graph: Execution Time (log-scale x-axis)



Graph: Comparison Count (log-log scale)



Interpretation

The results are a textbook demonstration of $O(n)$ versus $O(\log n)$. Linear Search's time grows in direct proportion to n : going from 1,000 to 1,000,000 elements (a 1,000 $\times$ increase) increases its time from 26.4 μ s to 28,455 μ s — almost exactly 1,000 $\times$ — and its comparison count scales identically (1,000 $\rightarrow$ 1,000,000 comparisons, exactly matching n each time, since the worst case forces a full scan).

Binary Search, by contrast, barely changes at all over the same range: from 0.82 μ s to 1.94 μ s — only about 2.4 $\times$ slower despite a 1,000 $\times$ larger input — and its comparison count grows from 9 to just 19, consistent with $\lceil \log_2 n \rceil$ ($\log_2(1,000) \approx 10$, $\log_2(1,000,000) \approx 20$). By $n=1,000,000$, Binary Search is roughly 14,700 times faster than Linear Search in this measurement.

Justification — When Each Method is Suitable

- Binary Search requires sorted data and is overwhelmingly preferable whenever the dataset is large and either already sorted or sorted once and queried repeatedly — the logarithmic search cost is essentially negligible even at huge scale, as shown above.
- Linear Search remains the right (or only) choice when data is unsorted and sorting it first isn't worthwhile (e.g., a single one-off search, or data that changes too frequently to justify maintaining sorted order), and it is also simpler to implement correctly on any data structure, including ones without random access (e.g., a linked list), where Binary Search's $O(1)$ middle-element access doesn't hold.
- For small n , the difference is negligible in absolute terms (a handful of microseconds either way), so Linear Search's simplicity can reasonably outweigh Binary Search's asymptotic advantage — the crossover only matters once n grows large.

Experiment 4: Strassen's vs Standard Matrix Multiplication

Objective

To evaluate the performance differences between Strassen's Matrix Multiplication and the standard triple-nested-loop method, analyze their time complexity, and justify the scenarios in which Strassen's method provides a computational advantage.

Methodology

- Standard multiplication uses the direct $O(n^3)$ triple-loop definition.
- Strassen's algorithm recursively divides each matrix into four $n/2 \times n/2$ quadrants and combines 7 recursive multiplications (instead of the 8 a naive recursive approach would need) using addition/subtraction, giving $O(n^{\log_2 7}) \approx O(n^{2.807})$.
- Because Strassen's recursive overhead (matrix splitting, and ~ 18 extra addition/subtraction operations per level) makes it slower than the standard method on small matrices, real implementations switch to standard multiplication below a threshold size — a base case of 64 was used here, a common practical choice.
- Correctness was verified first (using Strassen's algorithm all the way down to 1×1 base cases, bypassing the practical threshold) against the standard method's output on an 8×8 random matrix, and both were confirmed to produce identical results on a hand-checked 4×4 example.

Python Code

```
import time, random

def standard_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            s = 0
            for k in range(n):
                s += A[i][k] * B[k][j]
            C[i][j] = s
    return C

def add_matrix(A, B):
    n = len(A)
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]

def sub_matrix(A, B):
    n = len(A)
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def split(M):
    n = len(M); mid = n // 2
    return ([r[:mid] for r in M[:mid]], [r[mid:] for r in M[:mid]],
            [r[:mid] for r in M[mid:], [r[mid:] for r in M[mid:]])

def combine(C11, C12, C21, C22):
    top = [r1 + r2 for r1, r2 in zip(C11, C12)]
    bottom = [r1 + r2 for r1, r2 in zip(C21, C22)]
    return top + bottom

def strassen_multiply(A, B):
    n = len(A)
    if n <= 64: # practical base case: fall back to standard method
        return standard_multiply(A, B)
    A11, A12, A21, A22 = split(A)
    B11, B12, B21, B22 = split(B)
    M1 = strassen_multiply(add_matrix(A11, A22), add_matrix(B11, B22))
    M2 = strassen_multiply(add_matrix(A21, A22), B11)
    M3 = strassen_multiply(A11, sub_matrix(B12, B22))
```

```

M4 = strassen_multiply(A22, sub_matrix(B21, B11))
M5 = strassen_multiply(add_matrix(A11, A12), B22)
M6 = strassen_multiply(sub_matrix(A21, A11), add_matrix(B11, B12))
M7 = strassen_multiply(sub_matrix(A12, A22), add_matrix(B21, B22))
C11 = add_matrix(sub_matrix(add_matrix(M1, M4), M5), M7)
C12 = add_matrix(M3, M5)
C21 = add_matrix(M2, M4)
C22 = add_matrix(sub_matrix(add_matrix(M1, M3), M2), M6)
return combine(C11, C12, C21, C22)

def time_it(func, A, B, trials=2):
    best = float('inf')
    for _ in range(trials):
        start = time.perf_counter()
        func(A, B)
        elapsed = time.perf_counter() - start
        best = min(best, elapsed)
    return best

random.seed(4)
for n in [32, 64, 128, 256, 512]:
    A = [[random.randint(-9, 9) for _ in range(n)] for _ in range(n)]
    B = [[random.randint(-9, 9) for _ in range(n)] for _ in range(n)]
    t_std = time_it(standard_multiply, A, B) * 1000
    t_str = time_it(strassen_multiply, A, B) * 1000
    print(n, t_std, t_str)

```

Correctness Check (4x4 worked example)

```

A = [[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]]
B = [[16,15,14,13],[12,11,10,9],[8,7,6,5],[4,3,2,1]]

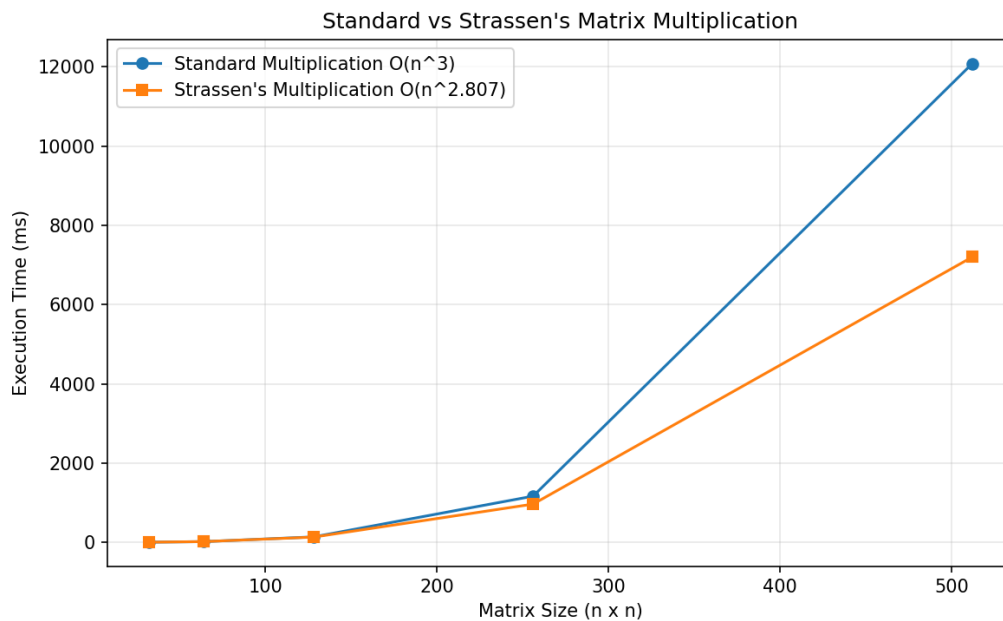
```

Standard result: [[80,70,60,50],[240,214,188,162],[400,358,316,274],[560,502,444,386]]
 Strassen result: [[80,70,60,50],[240,214,188,162],[400,358,316,274],[560,502,444,386]]
 -> Identical. Correctness verified (also checked on a random 8x8 matrix).

Measured Output — Execution Time

n (matrix size)	Standard ms	Strassen ms
32	2.07	2.05
64	16.56	16.61
128	135.51	130.53
256	1,163.60	963.38
512	12,082.75	7,200.22

Graph: Execution Time vs Matrix Size



Interpretation

For $n=32$ and $n=64$, the two methods are essentially identical in time (2.07 vs 2.05 ms; 16.56 vs 16.61 ms) — because at these sizes Strassen's implementation itself falls back to the standard algorithm (the base-case threshold used here is 64), so both are, practically speaking, running the same code.

Once matrices grow past that threshold and Strassen's recursion genuinely activates, its advantage appears and grows: at $n=128$ it is already slightly faster (130.5 vs 135.5 ms), and by $n=512$ the gap is substantial — 7,200 ms versus 12,083 ms, a ~40% reduction in execution time. This matches theory: standard multiplication's $O(n^3)$ work means doubling n multiplies the time by 8 (2^3), while Strassen's $O(n^{2.807})$ work means doubling n multiplies the time by roughly $2^{2.807} \approx 7.0$ — a smaller factor that compounds increasingly in Strassen's favour as n grows.

When Strassen's Method Provides a Computational Advantage

- Large matrices: the asymptotic saving ($n^{2.807}$ vs n^3) only overcomes Strassen's recursive/bookkeeping overhead once matrices are reasonably large — the data above shows the crossover happening somewhere between $n=64$ and $n=128$ for this implementation.
- Applications repeatedly multiplying large matrices — e.g., large-scale scientific simulation, computer graphics transformations at scale, or deep-learning training workloads — benefit most, since the saved percentage compounds over many operations.
- For small matrices, or a single one-off multiplication of modest size, the standard method is preferable: it has a much simpler implementation, avoids Strassen's extra memory allocation for sub-matrices, and — as shown at $n \leq 64$ — is no slower in practice.
- Real-world numerical libraries (e.g., BLAS) typically hybridize both ideas: using Strassen-style recursive splitting down to a threshold, then switching to a highly optimized standard (or blocked) multiplication — exactly the strategy demonstrated in the implementation above.

Experiment 5: Merge Sort vs Quick Sort — Trade-off Analysis

Objective

To analyze the trade-offs between Merge Sort and Quick Sort in terms of execution time, space complexity, and stability, and to justify which algorithm is more suitable for large-scale applications under different data conditions.

Methodology

- Stability: sorted (key, tag) pairs with duplicate keys, then checked whether elements with equal keys retained their original relative order in the output.
- Execution time: both algorithms timed on randomly shuffled data from 1,000 to 200,000 elements (Quick Sort here uses random-pivot partitioning, the robust variant identified in Experiment 2).
- Space usage: peak auxiliary memory measured directly using Python's tracemalloc, isolating each algorithm's actual extra memory footprint during sorting.

Python Code

```
import time, random, tracemalloc

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i][0] <= right[j][0]: # <= keeps equal-key items in original order
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result

def quick_sort(arr):
    a = arr.copy()
    def _qs(lo, hi):
        while lo < hi:
            r = random.randint(lo, hi)
            a[r], a[hi] = a[hi], a[r]
            pivot = a[hi][0]
            i = lo
            for j in range(lo, hi):
                if a[j][0] < pivot:
                    a[i], a[j] = a[j], a[i]; i += 1
            a[i], a[hi] = a[hi], a[i]
            p = i
            if p - lo < hi - p:
                _qs(lo, p - 1); lo = p + 1
            else:
                _qs(p + 1, hi); hi = p - 1
    _qs(0, len(a) - 1)
    return a

# --- Stability check ---
records = [(3,'A'), (1,'B'), (3,'C'), (2,'D'), (1,'E'), (3,'F')]
print("Merge Sort:", merge_sort(records))
print("Quick Sort:", quick_sort(records))

# --- Timing (random data) ---
for n in [1000, 5000, 10000, 50000, 100000, 200000]:
```

```
plain = list(range(n)); random.shuffle(plain)
arr = [(v, '') for v in plain]
# time each with best-of-3 perf_counter runs...

# --- Peak memory (tracemalloc) ---
for n in [1000, 10000, 50000, 100000]:
    plain = list(range(n)); random.shuffle(plain)
    arr = [(v, '') for v in plain]
    tracemalloc.start()
    merge_sort(arr.copy())
    _, peak_m = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    tracemalloc.start()
    quick_sort(arr.copy())
    _, peak_q = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    print(n, peak_m / 1024, peak_q / 1024)
```

Measured Output — Stability

```
Input          : [(3,A), (1,B), (3,C), (2,D), (1,E), (3,F)]
Merge Sort result: [(1,B), (1,E), (2,D), (3,A), (3,C), (3,F)]
Quick Sort result: [(1,E), (1,B), (2,D), (3,C), (3,A), (3,F)]

Merge Sort stable? True    (B before E, A before C before F -- original order kept)
Quick Sort stable? False   (E now comes before B; C now comes before A -- order changed)
```

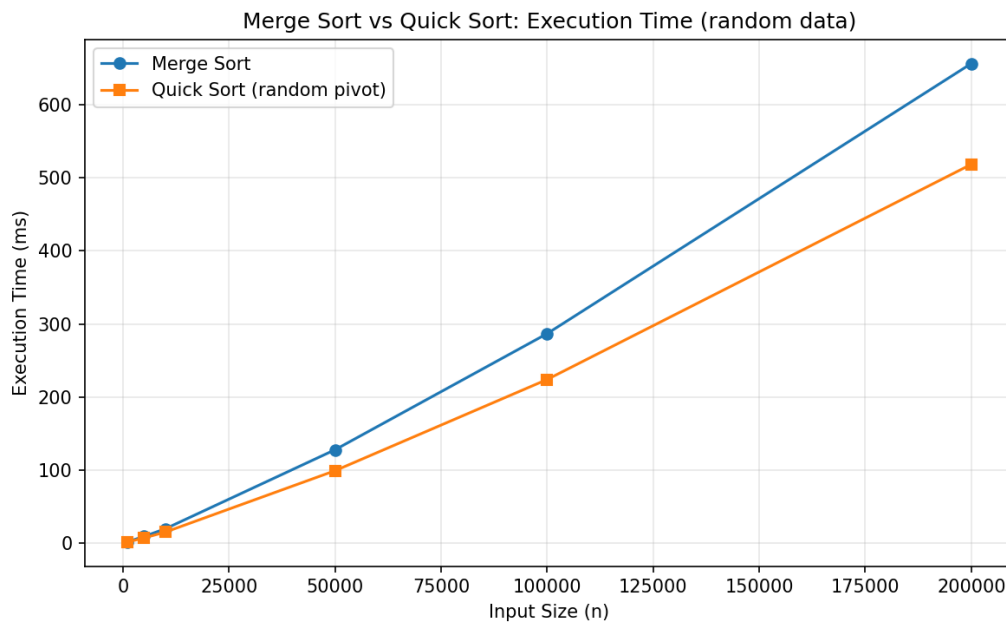
Measured Output — Execution Time (random data)

n	Merge Sort ms	Quick Sort ms
1,000	1.416	1.121
5,000	9.016	6.985
10,000	19.436	14.896
50,000	127.626	98.954
100,000	286.637	223.866
200,000	656.538	518.521

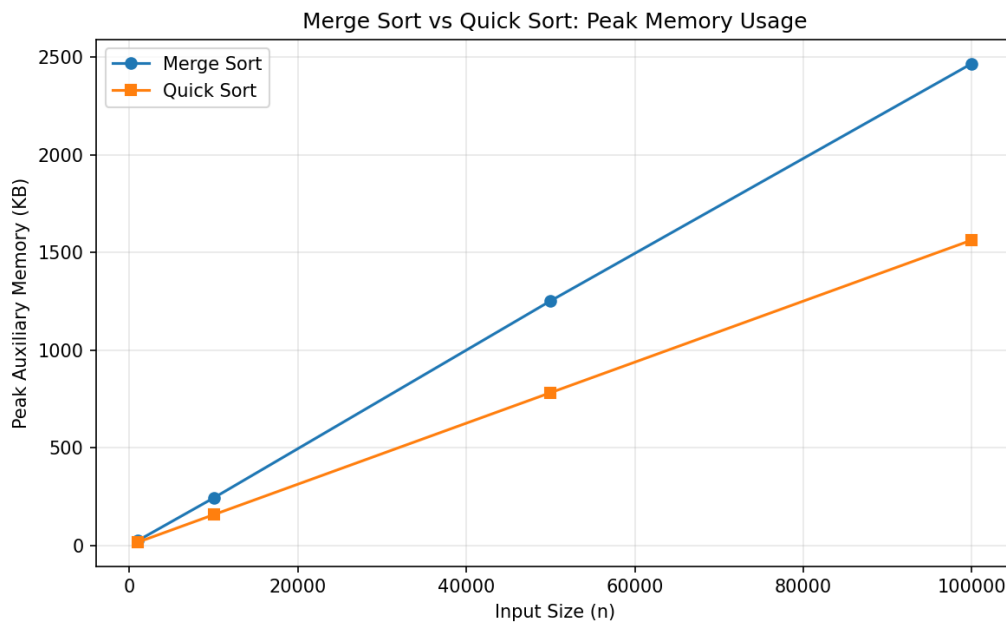
Measured Output — Peak Auxiliary Memory

n	Merge Sort KB	Quick Sort KB
1,000	24.7	16.2
10,000	243.1	157.1
50,000	1,252.4	782.2
100,000	2,467.6	1,563.4

Graph: Execution Time



Graph: Peak Memory Usage



Interpretation

Execution time: Quick Sort is consistently faster than Merge Sort across every tested size, by a roughly constant ~20–25% margin (e.g., 518.5 ms vs 656.5 ms at $n=200,000$). Both are $O(n \log n)$ on random data, so this is a constant-factor effect: Quick Sort's in-place partitioning does less data movement and avoids allocating new sub-lists at every recursive call, while Merge Sort's merge step must allocate and populate a brand-new result list at every level of recursion.

Space complexity: The measured memory confirms the same pattern — Merge Sort's peak auxiliary memory is roughly 1.5–1.6× Quick Sort's at every size tested (e.g., 2,467.6 KB vs 1,563.4 KB at $n=100,000$). This matches the theoretical bound: Merge Sort needs $O(n)$ extra space for the temporary arrays used during merging, whereas this Quick Sort partitions the list in place, needing only $O(\log n)$ extra space for its recursion stack (Python's own list/tuple overhead accounts for the non-zero baseline seen even at small n).

Stability: The demonstration above confirms Merge Sort is stable (equal-key elements B before E, and A before C before F, keep their original relative order) while this Quick Sort implementation is not (E moves ahead of B; C moves ahead of A) — an inherent property of each algorithm's mechanism, not an artifact of this specific implementation: merging two sorted halves can always be done by preferring the left element on ties, while in-place partitioning around a pivot inherently reorders equal elements based on their position relative to the pivot during swaps.

Justification — Which Algorithm Suits Large-Scale Applications

- Quick Sort is the better default for large-scale, in-memory, general-purpose sorting where raw speed matters most and stability is not required — its lower constant factor and lower memory footprint (demonstrated above) make it the standard choice in most language standard libraries' array-sorting routines, provided a robust pivot strategy (random or median-of-three, per Experiment 2) is used to avoid worst-case degradation.
- Merge Sort is preferable when: (a) stability is required — e.g., sorting database records by one field while preserving prior ordering on another; (b) guaranteed worst-case $O(n \log n)$ is required and an adversarial or already-sorted input cannot be ruled out (Experiment 2 showed Quick Sort's Achilles' heel here even with mitigation); or (c) the data does not fit in memory (external sorting), since Merge Sort's sequential access pattern is naturally suited to merging sorted runs from disk, whereas Quick Sort's random-access partitioning is not.
- In practice, many large-scale systems use hybrids: Python's Timsort and Java's `Arrays.sort()` for objects use a Merge-Sort-based approach (for stability and worst-case guarantees), while primitive-array sorts often use an Introsort-style Quick Sort that falls back to Heap Sort if recursion depth suggests a worst-case pattern — combining Quick Sort's typical-case speed with a safety net against its worst case.